Αναζήτηση εικόνων με διανυσματικές αναπαραστάσεις

Παντελής Κώτσας
sdi2300239

Σινάν Κιουτσούκ
sdi2300074

Περίληψη: **Αναπαριστούμε κάθε εικόνα με ιστόγραμμα $K = 256$ οπτικών λέξεων από τους SIFT descriptors της και συγκρίνουμε την εξαντλητική αναζήτηση με τις LSH, Hypercube, IVFFlat και IVFPQ στο HPatches, με έως 24985 εικόνες του MIRFlickr ως distractors. Οι παράμετροι επιλέγονται στο validation set, σε $D = 10000$, και η σύγκριση γίνεται στο test set για D από 1000 έως 25000. Με 25100 εικόνες στη βάση, η Exact Search δίνει mAP@10 0.690 σε 2.30 ms ανά query. Το IVFFlat δίνει πρακτικά την ίδια ποιότητα (0.690) σε 1.59 ms, το IVFPQ φτάνει τα 0.25 ms με mAP@10 0.668 και το LSH δίνει 0.684 σε 1.90 ms. Το Hypercube πέφτει στο 0.148 στη μεγαλύτερη βάση. Επαναλαμβάνοντας όλα τα τελικά πειράματα σε 5 τυχαία seeds, το Hypercube έχει τυπική απόκλιση στο mAP@10 από 4 έως 8 φορές μεγαλύτερη από κάθε άλλη μέθοδο, δηλαδή είναι και η λιγότερο σταθερή.**

I. Εισαγωγή

Η εργασία ζητά αναζήτηση εικόνων με βάση ένα διάνυσμα σταθερής διάστασης ανά εικόνα. Κάθε εικόνα μετατρέπεται σε ιστόγραμμα «οπτικών λέξεων» (Bag of Visual Words) από τους SIFT descriptors της [1], και πάνω σε αυτά τα διανύσματα συγκρίνουμε μία ακριβή και τέσσερις προσεγγιστικές μεθόδους αναζήτησης κοντινότερων γειτόνων. Ως queries χρησιμοποιούμε την πρώτη εικόνα κάθε ακολουθίας του HPatches [2], ενώ η βάση μεγαλώνει με εικόνες του MIRFlickr-25K [3] που λειτουργούν ως distractors.

Στην Ενότητα II περιγράφουμε την αναπαράσταση των εικόνων, στην Ενότητα III τις μεθόδους και τις σχεδιαστικές μας επιλογές, στην Ενότητα IV την πειραματική διάταξη και στην Ενότητα V τα αποτελέσματα, μαζί με μια επιπλέον δοκιμή ευρωστίας ως προς το τυχαίο seed που κάναμε πέρα από τα ζητούμενα της εκφώνησης. Η Ενότητα VI κλείνει με τα συμπεράσματα και τις δυσκολίες που συναντήσαμε.

Καταμερισμός εργασίας. Ο κώδικας είναι χωρισμένος σε δύο ανεξάρτητα μέρη. Ο Παντελής ανέλαβε τον αλγοριθμικό πυρήνα (`src/core`): το k-means και τις πέντε μεθόδους πίσω από την κοινή διεπαφή `SearchIndex`. Ο Σινάν ανέλαβε το pipeline (`src/pipeline`): εξαγωγή SIFT, λεξιλόγιο, cache των διανυσμάτων, παραμέτρους γραμμής εντολών, μετρικές και αρχείο εξόδου. Η διεπαφή μεταξύ των δύο μερών είναι μόνο ο πίνακας διανυσμάτων και η κλάση `SearchIndex`, οπότε κάθε μέρος γραφόταν και ελεγχόταν χωριστά. Δεν γράψαμε ξεχωριστό πρόγραμμα ελέγχου (`tests/test_core.cpp`): η εκφώνηση δεν το απαιτεί και προτιμήσαμε να προχωρήσουμε στα πειράματα με τον χρόνο που είχε απομείνει πριν την προθεσμία. Το σενάριο των πειραμάτων και η παρούσα αναφορά γράφτηκαν από κοινού, πάνω στα πραγματικά αρχεία εξόδου του `./search`.

II. Αναπαράσταση των εικόνων

Από κάθε εικόνα εξάγουμε όλους τους SIFT descriptors με την OpenCV, σε grayscale. Για το λεξιλόγιο διαλέγουμε τυχαία έως $S = 500$ descriptors από καθεμία από τις $70 \times 6 = 420$ εικόνες του training set (όλους, αν έχει λιγότερους) και τρέχουμε k-means με αρχικοποίηση k-means++. Τα K κέντρα που προκύπτουν χρησιμοποιούνται για όλες τις εικόνες. Για μια εικόνα I με n_I descriptors, κάθε descriptor αντιστοιχίζεται στο πλησιέστερο κέντρο και $x_I = (h_1/n_I, \ldots, h_K/n_I)$. Οι εικόνες validation, test και MIRFlickr δεν χρησιμοποιούνται στην εκπαίδευση του λεξιλογίου.

Το $K = 256$ το κρατήσαμε απευθείας από το παράδειγμα εκτέλεσης της εκφώνησης, χωρίς ξεχωριστή δοκιμή άλλων τιμών: η εκφώνηση δεν ζητά να βελτιστοποιηθεί το K, μόνο οι παράμετροι κάθε μεθόδου αναζήτησης, και με δεδομένο τον χρόνο που είχαμε προτιμήσαμε να επικεντρωθούμε εκεί. Για αυτό το K ισχύουν και οι προϋποθέσεις του IVFPQ, αφού το 256 διαιρείται με όλα τα M που δοκιμάσαμε.

III. Μέθοδοι αναζήτησης

Όλες οι μέθοδοι υλοποιούν την ίδια διεπαφή: `build` πάνω στα n διανύσματα της βάσης και `search`, που επιστρέφει έως 10 εικόνες ταξινομημένες κατά αύξουσα απόσταση. Η απόσταση είναι πάντα ευκλείδεια. Όλα τα τυχαία βήματα ξεκινούν από το `-seed`.

Α΄. Exact Search

Υπολογίζουμε την απόσταση του query από όλα τα διανύσματα της βάσης και κρατάμε τα 10 μικρότερα με `partial_sort`. Είναι η αναφορά για την ποιότητα των υπόλοιπων μεθόδων.

Β΄. LSH

Χρησιμοποιούμε τις συναρτήσεις $h(p) = \lfloor (v \cdot p + t)/w \rfloor$ με $v \sim \mathcal{N}(0, I)$ και t ομοιόμορφο στο $[0, w)$ [4]. Κάθε ένας από τους L πίνακες συνδυάζει k τέτοιες συναρτήσεις σε ένα κλειδί με τυχαίους περιττούς πολλαπλασιαστές, και χρησιμοποιεί αυτό το κλειδί ως κάδο. Έτσι ένας κάδος περιέχει μόνο σημεία που συμφωνούν σε όλες τις k τιμές. Οι υποψήφιοι είναι η ένωση των L κάδων του query, χωρίς διπλότυπα, και κατατάσσονται με την πραγματική απόσταση.

Γ΄. Hypercube

Κάθε σημείο προβάλλεται σε κορυφή του $\{0,1\}^{d'}$: το bit j προκύπτει από την h_j (ίδιας μορφής με το LSH) μέσω ενός τυχαίου «νομίσματος» που εξαρτάται από το ζεύγος $(j, h_j(p))$. Η αναζήτηση εξετάζει πρώτα την κορυφή του query και μετά τις γειτονικές κατά αύξουσα απόσταση Hamming (οι συνδυασμοί bits παράγονται με το τέχνασμα του Gosper), μέχρι να ελεγχθούν `probes` κορυφές ή M σημεία.

Δ΄. IVFFlat

Τρέχουμε k-means στα n διανύσματα της βάσης (διαφορετικό από το k-means του λεξιλογίου) με $\lceil\sqrt{n}\rceil$ συστάδες ως default, και κρατάμε μία λίστα ανά συστάδα. Το query συγκρίνεται με τα κέντρα και σαρώνει εξαντλητικά τις λίστες των `nprobe` πλησιέστερων συστάδων.

Ε΄. IVFPQ

Ο coarse quantizer είναι ο ίδιος με του IVFFlat. Για κάθε διάνυσμα κρατάμε το residual από το κέντρο της συστάδας του, το χωρίζουμε σε M υποδιανύσματα διάστασης K/M και κβαντίζουμε κάθε υποδιάνυσμα σε ένα από 2^{nbits} κέντρα, που μαθαίνονται με k-means ανά υποχώρο [5]. Στην αναζήτηση, για κάθε συστάδα που ελέγχεται υπολογίζουμε το residual του query και έναν πίνακα $M \times 2^{nbits}$ με τις αποστάσεις των υποδιανυσμάτων του από όλα τα κέντρα (Asymmetric Distance Computation). Η απόσταση από κάθε αποθηκευμένη εικόνα είναι άθροισμα M τιμών του πίνακα, χωρίς re-ranking.

IV. Πειραματική διάταξη

Δεδομένα. Από τις 116 ακολουθίες του HPatches, 70 είναι στο training set, 23 στο validation (11 illumination, 12 viewpoint) και 23 στο test (11 και 12). Για κάθε σύνολο queries η βάση περιέχει τις εικόνες $I_2, \ldots, I_6$ των 23 ακολουθιών του (115 εικόνες) και τις πρώτες $D \in \{1000, 5000, 10000, 25000\}$ εικόνες του MIRFlickr κατά αύξοντα αριθμό.

Δεκαπέντε εικόνες του MIRFlickr δεν δίνουν κανέναν SIFT descriptor (π.χ. `im2009.jpg` και `im13064.jpg`) και τις παραλείπουμε όπως ορίζει η εκφώνηση, παίρνοντας την επόμενη. Για $D = 25000$ δεν υπάρχουν αρκετές επόμενες εικόνες στο αρχείο: η βάση καταλήγει με 24985 εικόνες MIRFlickr, άρα 25100 συνολικά αντί για 25115.

Μετρικές και χρόνος. Υπολογίζουμε Recall@5, Recall@10 και AP@10 ανά query όπως στην εκφώνηση (με παρονομαστή πάντα 5) και τους μέσους όρους τους. Αν μια μέθοδος επιστρέψει λιγότερες από 10 εικόνες, οι θέσεις που λείπουν μετράνε ως μη σχετικές. Ο χρόνος μετράει μόνο την κλήση `search` για κάθε query, σε ένα νήμα. Η εξαγωγή SIFT, τα διανύσματα και η κατασκευή των ευρετηρίων δεν μετράνε στον χρόνο αναζήτησης, παρόλο που την αναφέρουμε ξεχωριστά στην Ενότητα V-Γ΄. Όλες οι μετρήσεις έγιναν σε macOS (Darwin 25.6.0, arm64) πάνω σε Apple M1 Max με 10 πυρήνες, με τον μεταγλωττιστή Apple clang 21 (μέσω της εντολής `g++`) και `-O3`, και OpenCV 5.0.0. Με 23 queries ανά σύνολο, κάθε query αντιστοιχεί σε περίπου 0.043 στο mAP@10, οπότε μικρές διαφορές ανάμεσα στις μεθόδους δεν είναι αξιόπιστες.

Πίνακας I
Παράμετροι που επιλέχθηκαν στο validation set, $D = 10000$

Μέθοδος	Παράμετροι	mAP@10
LSH	$w = 0.2$	0.6684
Hypercube	$w = 0.1$	0.3696
IVFFlat	$kclusters = 300$, $nprobe = 20$	0.7388
IVFPQ	$M = 32$	0.7068

Επιλογή παραμέτρων. Για κάθε προσεγγιστική μέθοδο τρέξαμε στο validation set, σε $D = 10000$, ένα πλέγμα από 7 έως 8 ρυθμίσεων γύρω από τις default τιμές (αλλάζοντας μία ή δύο παραμέτρους κάθε φορά), συνολικά 30 ρυθμίσεις. Κρατήσαμε τη ρύθμιση με το μεγαλύτερο mAP@10 και τη χρησιμοποιήσαμε για όλα τα D στο test set. Αυτό είναι μια απλούστευση σε σχέση με το να ξαναγίνεται η επιλογή για κάθε D ξεχωριστά, αλλά μας επέτρεψε να τρέξουμε τα 20 τελικά πειράματα, και αργότερα την επανάληψή τους σε 5 seeds, στον χρόνο που είχαμε. Όλη η διαδικασία έγινε με ένα σενάριο bash (`scripts/run_experiments.sh`) που τρέχει το `./search` και μαζεύει τα αποτελέσματα σε CSV.

V. Αποτελέσματα

Α΄. Επιλεγμένες παράμετροι

Ο Πίνακας I δείχνει τις ρυθμίσεις που κράτησε το πλέγμα στο validation set, $D = 10000$. Στο IVFFlat τέσσερις ρυθμίσεις έδωσαν το ίδιο mAP@10 (0.7388): το default, `nprobe`=10, `nprobe`=20 και `kclusters`=50. Κρατήσαμε `kclusters`=300 με `nprobe`=20, μια από τις ισοδύναμες.

Β΄. Σύγκριση στο test set

Ο Πίνακας II συγκεντρώνει τα αποτελέσματα του test set και τα Σχήματα 1 και 2 δείχνουν την ποιότητα και τον χρόνο συναρτήσει του D. Μόνο ένα query πήρε λιγότερα από 10 αποτελέσματα (Hypercube, $D = 1000$).

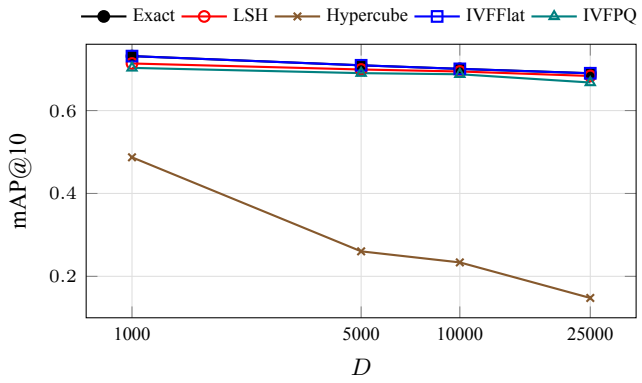
Η Exact Search χάνει ποιότητα όσο μεγαλώνει η βάση: το mAP@10 πέφτει από 0.731 για $D = 1000$ σε 0.709 για $D = 5000$ και σε 0.690 για $D = 25000$. Ο χρόνος της αυξάνεται περίπου γραμμικά με το n (0.10, 0.46, 0.91 και 2.30 ms).

Το IVFFlat είναι η πιο σταθερή προσεγγιστική μέθοδος: δίνει ακριβώς το ίδιο mAP@10 με την Exact Search σε κάθε D, και είναι γρηγορότερο σε όλα (1.2 έως 1.4 φορές), εκτός από $D = 1000$ όπου η διαφορά είναι αμελητέα. Αυτό συμβαίνει γιατί το `nprobe`=20 με `kclusters`=300 καλύπτει σχεδόν όλη τη βάση σε αυτό το μέγεθος, οπότε η μέθοδος πλησιάζει την εξαντλητική αναζήτηση στην ποιότητα.

Το IVFPQ είναι το γρηγορότερο για τη μεγαλύτερη βάση: για $D = 25000$ δίνει 0.668 σε 0.25 ms, περίπου 9 φορές γρηγορότερα από την Exact Search, με απώλεια 0.022 σε mAP@10 σε σχέση με το IVFFlat. Για μικρή βάση όμως δεν κερδίζει χρόνο: για $D = 1000$ είναι πιο αργό από την Exact Search (0.14 έναντι 0.10 ms), γιατί ο πίνακας αποστάσεων $M \times 256$ που υπολογίζει ανά συστάδα κοστίζει όσο αρκετές πλήρεις συγκρίσεις, κάτι που αποσβένεται μόνο με μεγάλες λίστες.

Πίνακας II
Αποτελέσματα στο test set (23 queries). Χρόνος σε ms ανά query, <10: queries με λιγότερα από 10 αποτελέσματα

D	Μέθοδος	R@5	R@10	mAP@10	χρόνος	<10
1000	Exact	0.7217	0.7826	0.7314	0.0994	0
1000	LSH	0.7217	0.7565	0.7136	0.0873	0
1000	Hypercube	0.4870	0.5130	0.4872	0.0195	1
1000	IVFFlat	0.7217	0.7826	0.7314	0.0838	0
1000	IVFPQ	0.6957	0.7652	0.7030	0.1410	0
5000	Exact	0.7043	0.7565	0.7092	0.4576	0
5000	LSH	0.7043	0.7304	0.6991	0.3934	0
5000	Hypercube	0.2609	0.2870	0.2603	0.0475	0
5000	IVFFlat	0.7043	0.7565	0.7092	0.2618	0
5000	IVFPQ	0.6783	0.7304	0.6903	0.1755	0
10000	Exact	0.6957	0.7478	0.7006	0.9129	0
10000	LSH	0.6957	0.7304	0.6943	0.7564	0
10000	Hypercube	0.2348	0.2435	0.2336	0.0827	0
10000	IVFFlat	0.6957	0.7478	0.7006	0.6744	0
10000	IVFPQ	0.6696	0.7217	0.6876	0.2005	0
25000	Exact	0.6870	0.7217	0.6900	2.2972	0
25000	LSH	0.6783	0.7130	0.6835	1.8970	0
25000	Hypercube	0.1478	0.1478	0.1478	0.1053	0
25000	IVFFlat	0.6870	0.7217	0.6900	1.5876	0
25000	IVFPQ	0.6522	0.6957	0.6677	0.2470	0

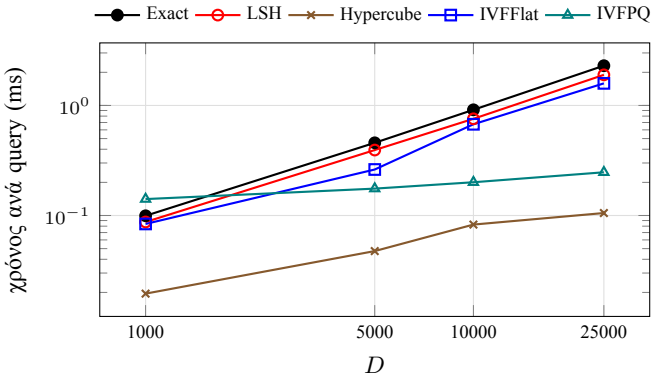


Σχήμα 1. mAP@10 στο test set συναρτήσει του πλήθους D των distractors

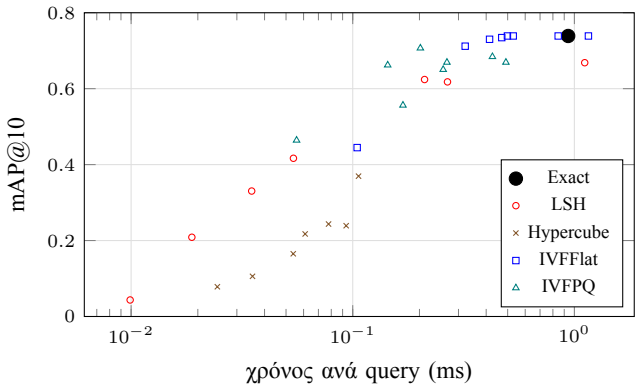
Το LSH δίνει 0.684 για $D = 25000$, κοντά στην Exact Search, και είναι 1.2 φορές γρηγορότερο. Το Hypercube είναι η πιο αδύναμη μέθοδος σε όλα τα D και η πτώση είναι απότομη: 0.487 για $D = 1000$, 0.234 για $D = 10000$ και 0.148 για $D = 25000$, παρόλο που ο χρόνος του μένει πολύ μικρός (0.02 έως 0.11 ms). Με $w = 0.1$ και τις default $kproj = 10$, $M = 500$, $probes = 10$ οι κορυφές κοντά στο query δεν καλύπτουν αρκετά σημεία ώστε να βρεθούν οι σχετικές εικόνες όσο μεγαλώνει η βάση.

Γ΄. Ποιότητα και χρόνος

Το Σχήμα 3 δείχνει όλες τις ρυθμίσεις του πλέγματος στο validation set, $D = 10000$, μαζί με το σημείο της Exact Search. Το IVFPQ και το IVFFlat είναι οι πιο αριστερά καμπύλες: φτάνουν υψηλό mAP@10 με πολύ λιγότερο χρόνο από το LSH, που χρειάζεται πάνω από 1 ms για να πλησιάσει την Exact Search (0.939 ms, 0.7388). Το Hypercube δεν ξεπερνά το 0.370 σε καμία ρύθμιση του πλέγματος.



Σχήμα 2. Μέσος χρόνος αναζήτησης στο test set (λογαριθμικοί άξονες)



Σχήμα 3. Όλες οι ρυθμίσεις του πλέγματος στο validation set, $D = 10000$

Πέρα από τον χρόνο αναζήτησης, οι μέθοδοι διαφέρουν πολύ στην κατασκευή του ευρετηρίου. Για $D = 25000$ με τις επιλεγμένες ρυθμίσεις, το Hypercube χτίζεται σε 55 ms και το LSH σε 114 ms, ενώ το IVFFlat χρειάζεται 3.6 s και το IVFPQ 7.2 s, λόγω του k-means πάνω στις 25100 βάσεις και, στο IVFPQ, του επιπλέον k-means ανά υποδιάνυσμα.

Δ΄. Illumination και viewpoint

Ο Πίνακας III χωρίζει το mAP@10 του test set στις 11 ακολουθίες illumination και στις 12 viewpoint. Για την Exact Search και το IVFFlat οι δύο κατηγορίες είναι κοντά σε κάθε D (π.χ. 0.695 και 0.686 για $D = 25000$). Το LSH χάνει λίγο περισσότερο στις illumination ακολουθίες σε μεγάλα D (0.666 έναντι 0.699 viewpoint, $D = 25000$). Το Hypercube είναι η μέθοδος που χάνει περισσότερο και στις δύο κατηγορίες καθώς μεγαλώνει το D, με τις illumination ακολουθίες να πέφτουν πιο απότομα (0.091 για $D = 25000$, έναντι 0.200 στις viewpoint). Με 11 ή 12 queries ανά κατηγορία δεν βγάζουμε γενικό συμπέρασμα από αυτή τη διαφορά, μόνο την καταγράφουμε.

Ε΄. Ευρωστία ως προς το τυχαίο seed

Πέρα από τα ζητούμενα της εκφώνησης, ξανατρέξαμε όλα τα 20 τελικά πειράματα (5 μέθοδοι επί 4 τιμές του D) σε 5 διαφορετικά seeds (1 έως 5), δηλαδή 100 εκτελέσεις συνολικά,

Πίνακας III
mAP@10 στο test set ανά τύπο ακολουθίας (Ill.: 11 queries illumination, View.: 12 queries viewpoint)

	$D = 1000$		$D = 5000$		$D = 10000$		$D = 25000$	
Μέθοδος	Ill.	View.	Ill.	View.	Ill.	View.	Ill.	View.
Exact	0.7125	0.7487	0.7095	0.7089	0.7000	0.7012	0.6945	0.6858
LSH	0.6780	0.7462	0.6709	0.7249	0.6697	0.7168	0.6664	0.6992
Hypercube	0.4576	0.5144	0.1848	0.3295	0.1673	0.2944	0.0909	0.2000
IVFFlat	0.7125	0.7487	0.7095	0.7089	0.7000	0.7012	0.6945	0.6858
IVFPQ	0.7066	0.6998	0.6879	0.6925	0.7136	0.6637	0.6652	0.6700

Πίνακας IV
Μέση τυπική απόκλιση σε 5 seeds (μέσος όρος στα 4 D)

Μέθοδος	std(mAP@10)	std(χρόνος, ms)
Exact	0.0070	0.1848
LSH	0.0106	0.1166
Hypercube	0.0565	0.0261
IVFFlat	0.0077	0.1177
IVFPQ	0.0149	0.0622

για να δούμε πόσο σταθερά είναι τα αποτελέσματα. Το seed επηρεάζει το k-means του λεξιλογίου, άρα και τα ίδια τα διανύσματα, και όχι μόνο το k-means του IVFFlat και του IVFPQ.

Ο Πίνακας IV δείχνει τη μέση τυπική απόκλιση του mAP@10 και του χρόνου ανά query στα 5 seeds, με μέσο όρο στα 4 D. Το Hypercube έχει τυπική απόκλιση στο mAP@10 (0.057) από 4 έως 8 φορές μεγαλύτερη από κάθε άλλη μέθοδο, δηλαδή είναι όχι μόνο η πιο αδύναμη αλλά και η λιγότερο σταθερή: μικρές αλλαγές στο λεξιλόγιο αλλάζουν αρκετά ποιες κορυφές του υπερκύβου βρίσκονται κοντά στο query. Το IVFFlat έχει σχεδόν την ίδια διακύμανση με την Exact Search (0.0077 έναντι 0.0070), που επιβεβαιώνει ότι οι δύο μέθοδοι δίνουν ουσιαστικά το ίδιο αποτέλεσμα και όχι μόνο στο seed=1 που χρησιμοποιήσαμε στον Πίνακα II. Η τυπική απόκλιση του χρόνου είναι μεγαλύτερη στην Exact Search και στο IVFFlat (περίπου 0.12 έως 0.18 ms) παρά στο IVFPQ και στο Hypercube, πιθανώς επειδή κάθε seed διαβάζει ξεχωριστό αρχείο cache από τον δίσκο και η καθυστέρηση ανάγνωσης ποικίλλει, όχι επειδή ο ίδιος ο αλγόριθμος αναζήτησης έχει τόση διακύμανση.

VI. Συμπεράσματα

Η αναπαράσταση BoVW με $K = 256$ βρίσκει τις σχετικές εικόνες αρκετά καλά (mAP@10 0.69 έως 0.73 με την Exact Search), αλλά η ποιότητα πέφτει όσο προστίθενται distractors. Ανάμεσα στις προσεγγιστικές μεθόδους, το IVFFlat είναι η ασφαλέστερη επιλογή, γιατί με τις ρυθμίσεις που επιλέξαμε ταυτίζεται με την Exact Search σε κάθε D και είναι πάντα το ίδιο ή γρηγορότερο. Το IVFPQ κερδίζει πολύ περισσότερο χρόνο στη μεγάλη βάση με μικρή απώλεια ποιότητας, και χρειάζεται πολύ λιγότερη μνήμη ανά εικόνα. Το LSH μένει κοντά στην Exact Search αλλά χωρίς να κερδίζει πολύ χρόνο, ενώ το Hypercube με τις default παραμέτρους του $d' = 10$ δεν αρκεί για 25100 σημεία και είναι η μόνη μέθοδος που χάνει σημαντικά τόσο σε ποιότητα όσο και σε σταθερότητα ανάμεσα σε seeds.

Οι δυσκολίες που συναντήσαμε ήταν κυρίως στο πρόγραμμα, όχι στα δεδομένα. Το `include/pipeline/cli/cli.hpp` είχε αρχικά το `enum class Method` δηλωμένο μέσα στο `struct Options`, οπότε κάθε χρήση του ονόματος `Method` έξω από το struct δεν μεταγλωττιζόταν· το μετακινήσαμε στο επίπεδο του αρχείου. Το `src/pipeline/dataset/database.cpp` είχε όλες τις βοηθητικές συναρτήσεις του (`cacheKey`, `loadOrTrainVocabulary` κ.λπ.) αλλά όχι την ίδια τη `buildCollections` που τις καλεί, πράγμα που φαινόταν και από τις προειδοποιήσεις «unused function» και από σφάλμα στο linking· τη συμπληρώσαμε. Το ότι το κλειδί της cache δεν εξαρτάται από το D, και η cache του MIRFlickr μεγαλώνει σταδιακά αντί να ξαναχτίζεται, μας επέτρεψε να τρέξουμε τα περίπου 150 πειράματα (πλέγμα, τελικά, 5 seeds) σε λίγα λεπτά συνολικά, τρέχοντας πάντα από το μικρότερο στο μεγαλύτερο D με το ίδιο `-cache`. Στα 100 πειράματα των 5 seeds είχαμε και δύο πρακτικά λάθη δικά μας: ένα σενάριο πειραμάτων που αφήσαμε κατά λάθος να τρέχει σε δύο αντίγραφα ταυτόχρονα έγραψε στα ίδια αρχεία cache και έδωσε ασυνεπείς διπλότυπες γραμμές στα αποτελέσματα, οπότε καθαρίσαμε την cache και ξανατρέξαμε καθαρά· και ένα `printf` στο σενάριο συγκέντρωσης των αποτελεσμάτων είχε 7 `%.4f` αντί για 8, οπότε η τυπική απόκλιση του χρόνου έλειπε σιωπηλά από κάθε γραμμή μέχρι να το προσέξουμε.

Αναφορές

[1] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vision*, vol. 60, no. 2, pp. 91-110, 2004.

[2] V. Balntas, K. Lenc, A. Vedaldi and K. Mikolajczyk, "HPatches: A benchmark and evaluation of handcrafted and learned local descriptors," in *Proc. CVPR*, 2017.

[3] M. J. Huiskes and M. S. Lew, "The MIR Flickr retrieval evaluation," in *Proc. ACM MIR*, 2008.

[4] M. Datar, N. Immorlica, P. Indyk and V. S. Mirrokni, "Locality-sensitive hashing scheme based on p-stable distributions," in *Proc. SoCG*, 2004.

[5] H. Jégou, M. Douze and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. PAMI*, vol. 33, no. 1, pp. 117-128, 2011.